Projet Compilation

Etape #1

Groupe 5 L3 MIASHS, parcours MIAGE, 2025-2026 :

BABACHANAKH Kateryna
SHKRED Diana
CIFTCI Mehmet Can
SAIDANI Alyssa

Table des matières

Introduction 3
Exemple 1 : Programme minimal 4
Exemple 2 : Variables globales 6
Exemple 3 : Expression 8
Exemple 4 : Ecriture 11
Exemple 5 : Ecriture 15
Exemple 6 : Variables locales et paramètres 18
Exemple 7 : Conditionnelle 22
Exemple 8 : Itération 25
Exemple 9 : Récursivité 29
Conclusion 33

Introduction

Dans le cadre du projet de compilation (UE 602 – L3 MIASHS), cette première étape consiste à développer le module de génération de code assembleur BETA à partir d'un arbre syntaxique abstrait. L'objectif est de transformer un programme de notre mini-langage en code exécutable sous BSim.

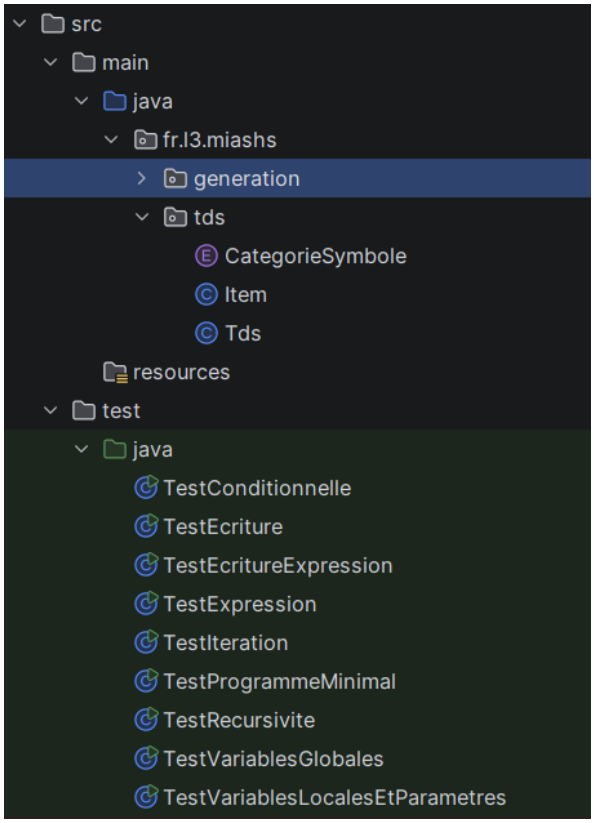
Le projet est disponible sur notre dépôt GitHub :

https://github.com/babachanakh-kateryna/project-compilexxl-l3.git

Le package ARBRE, fourni par l'enseignant, a été intégré au projet via les dépendances du fichier pom.xml (projet Maven).

La structure principale du projet comprend :

- *fr.l3.miashs.generation* : génération du code assembleur,
- *fr.l3.miashs.tds* : gestion de la table des symboles,
- *test* : classes de test correspondant aux différents exemples étudiés.

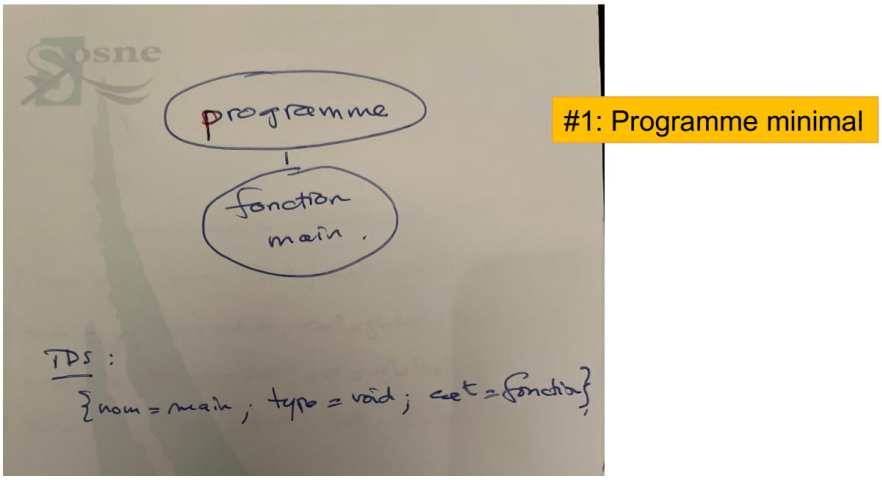


Cette étape regroupe plusieurs exemples progressifs (programme minimal, variables, expressions, conditionnelles, itérations, récursivité) validés par exécution sous BSim.

Exemple 1 : Programme minimal

1. Condition du test

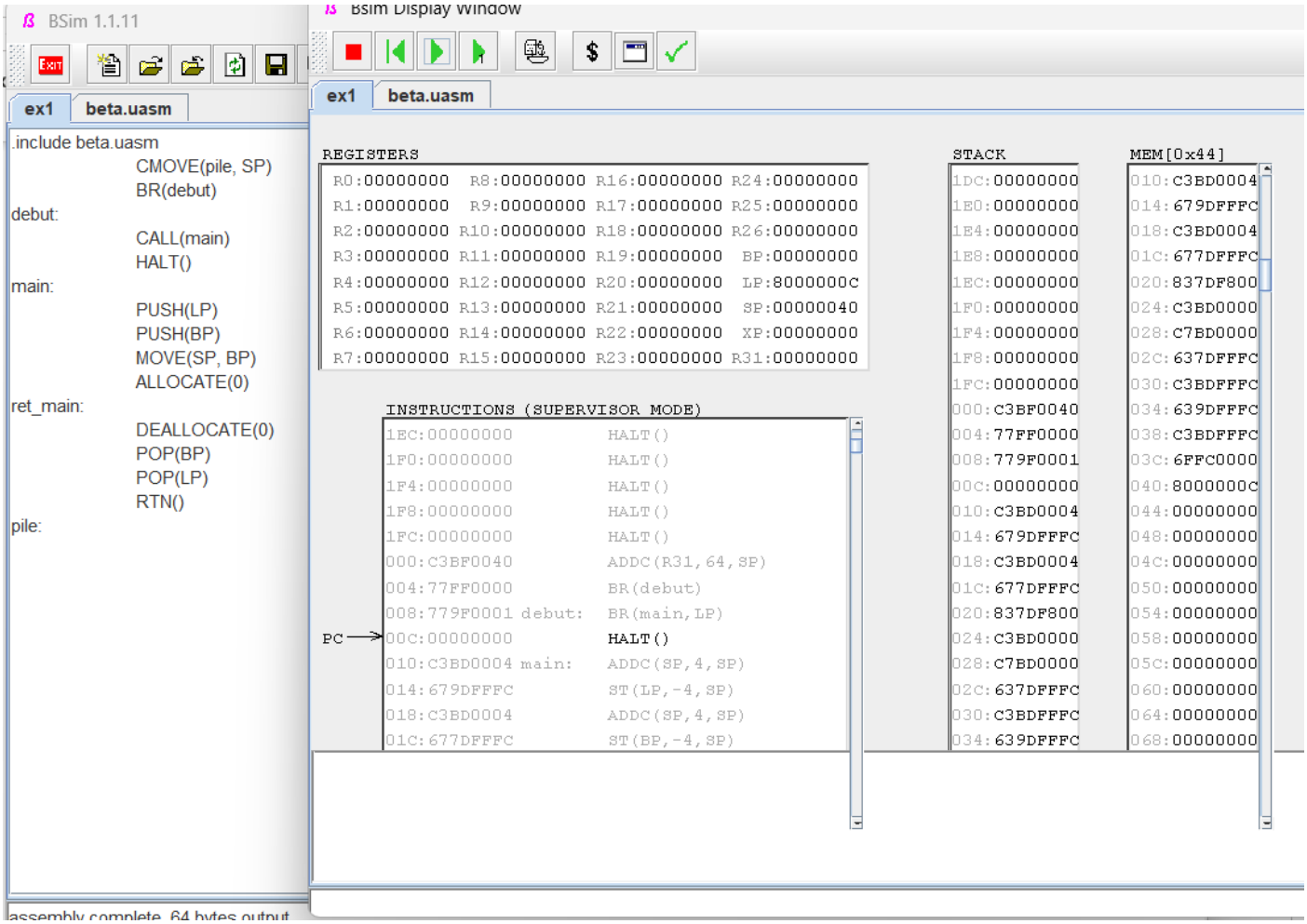
Programme composé uniquement d'une fonction main sans paramètres et sans instructions.



2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }

========== CODE GENERE ==========
.include beta.uasm
        CMOVE(pile, SP)
        BR(debut)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Le code généré correspond au comportement attendu pour un programme minimal :

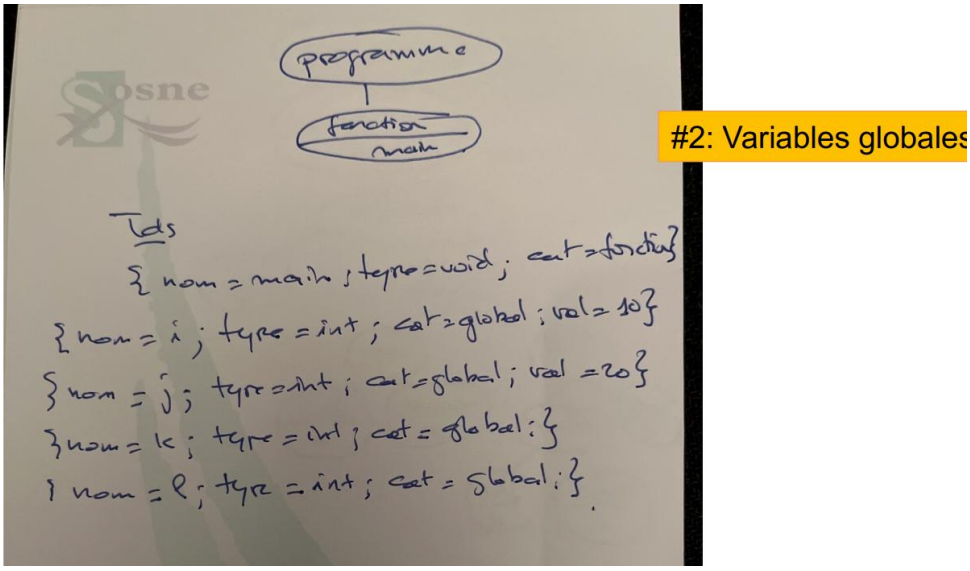
1. la fonction *main* est correctement appelée,
2. le prologue (sauvegarde de LP et BP) et l'épilogue (restauration de la pile) sont bien générés,
3. la pile est correctement restaurée,
4. le programme se termine par *HALT()* sans erreur.

L'exécution dans BSim confirme que le programme fonctionne correctement.

Exemple 2 : Variables globales

1. Condition du test

Programme minimal avec une fonction *main* et quatre variables globales (deux initialisées et deux non initialisées).

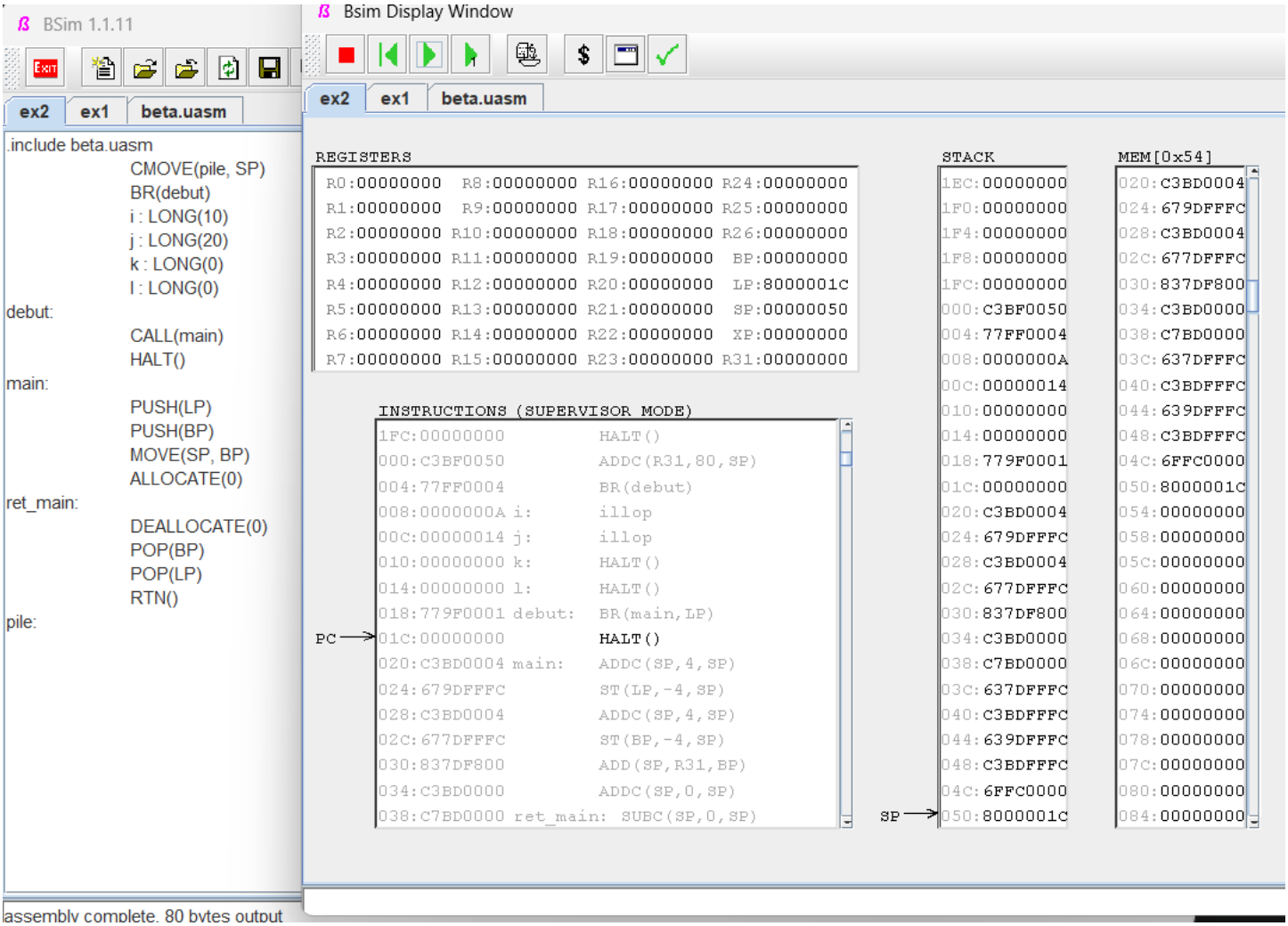


2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=i, type=int, cat=GLOBAL, val=10 }
{ nom=j, type=int, cat=GLOBAL, val=20 }
{ nom=k, type=int, cat=GLOBAL }
{ nom=l, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
        CMOVE(pile, SP)
        BR(debut)
        i : LONG(10)
        j : LONG(20)
        k : LONG(0)
        l : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
```

```
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
```

3. Exécution avec BSim



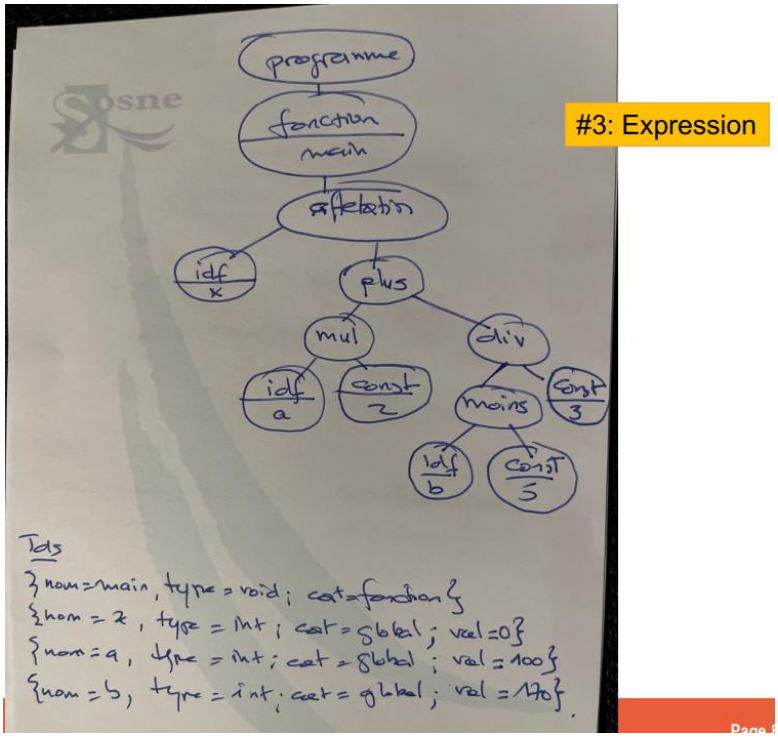
4. Évaluation : comparaison avec le code attendu

Le code généré correspond au code attendu. Les variables globales sont correctement déclarées dans la section des données avec l'initialisation appropriée pour *i* et *j*, tandis que *k* et *l* sont initialisées à 0 par défaut. La fonction *main* contient un prologue et un épilogue conformes à la convention d'appel vue en cours. L'exécution sous BSim ne produit aucune erreur, ce qui confirme la validité de la génération.

Exemple 3 : Expression

1. Condition du test

Programme contenant une fonction *main* avec une instruction d'affectation : $x=a*2+(b-5)/3$. Variables globales : x = 0, a = 100, b = 170.

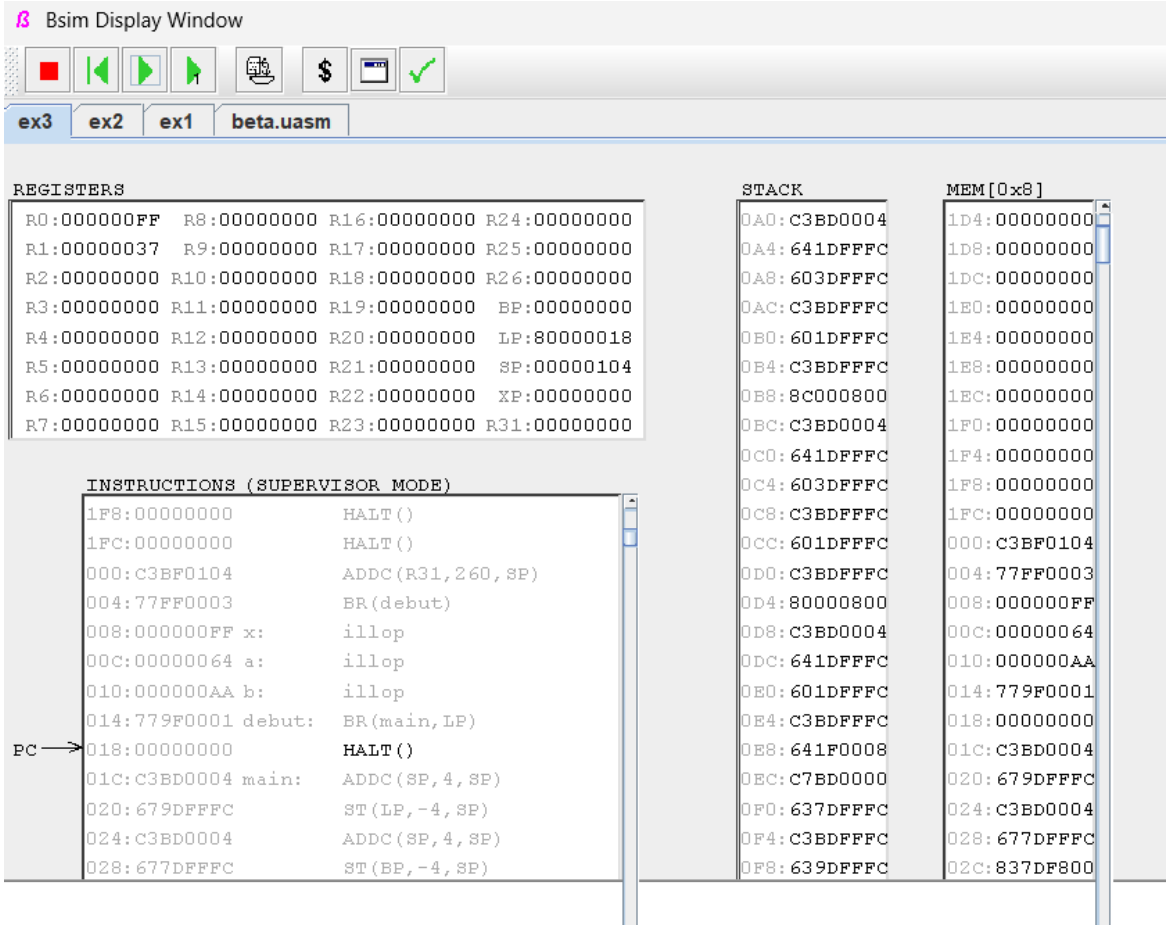


2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
    └─AFF
        └─IDF/x
        └─PLUS
            └─MUL
                └─IDF/a
                └─CONST/2
            └─DIV
                └─MOINS
                    └─IDF/b
                    └─CONST/5
                └─CONST/3

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=x, type=int, cat=GLOBAL, val=0 }
{ nom=a, type=int, cat=GLOBAL, val=100 }
{ nom=b, type=int, cat=GLOBAL, val=170 }
```

```
========== CODE GENERE ==========
.include beta.uasm
        CMOVE(pile, SP)
        BR(debut)
        x : LONG(0)
        a : LONG(100)
        b : LONG(170)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        LD(a, R0)
PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        LD(b, R0)
PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Le code généré respecte la priorité des opérations et la structure de l'arbre abstrait. Les instructions arithmétiques sont correctement ordonnées, la gestion de la pile est cohérente et le résultat final est bien sauvegardé dans la variable globale *x*. L'exécution dans BSim confirme que la valeur obtenue est 255, ce qui correspond exactement au résultat attendu.

La vérification :

Calcul mathématique :

1. a * 2 = 100 * 2 = 200
2. b - 5 = 170 - 5 = 165
3. 165 / 3 = 55
4. 200 + 55 = 255

Après exécution complète du programme dans BSim, on obtient :

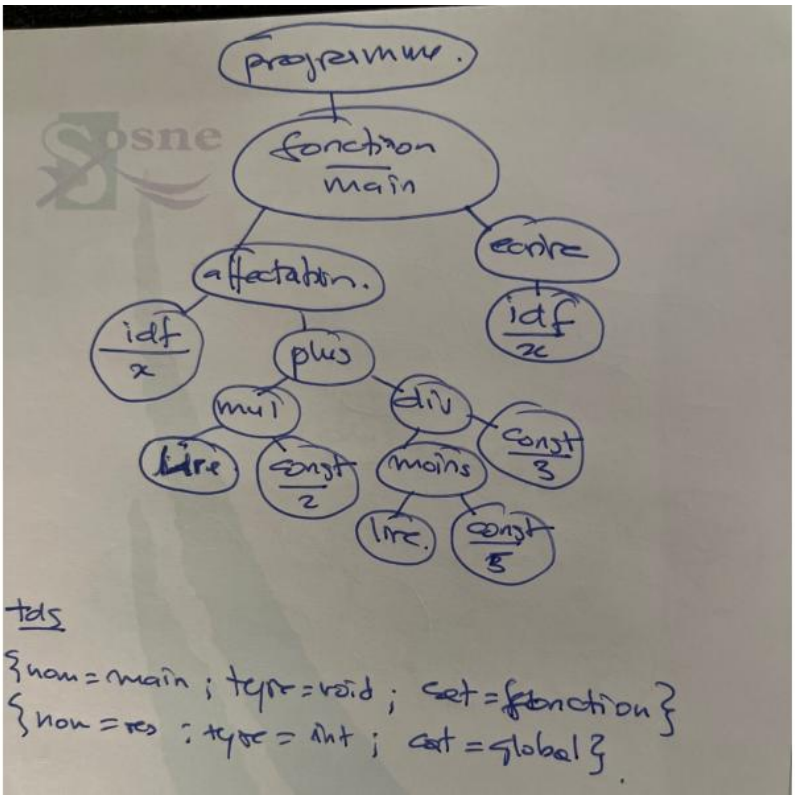
1. x (0x008) = 000000FF
2. a (0x00C) = 00000064
3. b (0x010) = 000000AA

255 en hexadécimal = 0xFF => La valeur stockée en mémoire correspond donc au résultat mathématique attendu.

Exemple 4 : Ecriture

1. Condition du test

Programme contenant une fonction *main* sans paramètres avec une variable globale *x*. Le programme lit deux entiers au clavier, calcule l'expression *x = lire()*2 + (lire()-5)/3*, puis affiche la valeur de *x* à l'aide de l'instruction *ecrire(x)*.



#4: Ecriture

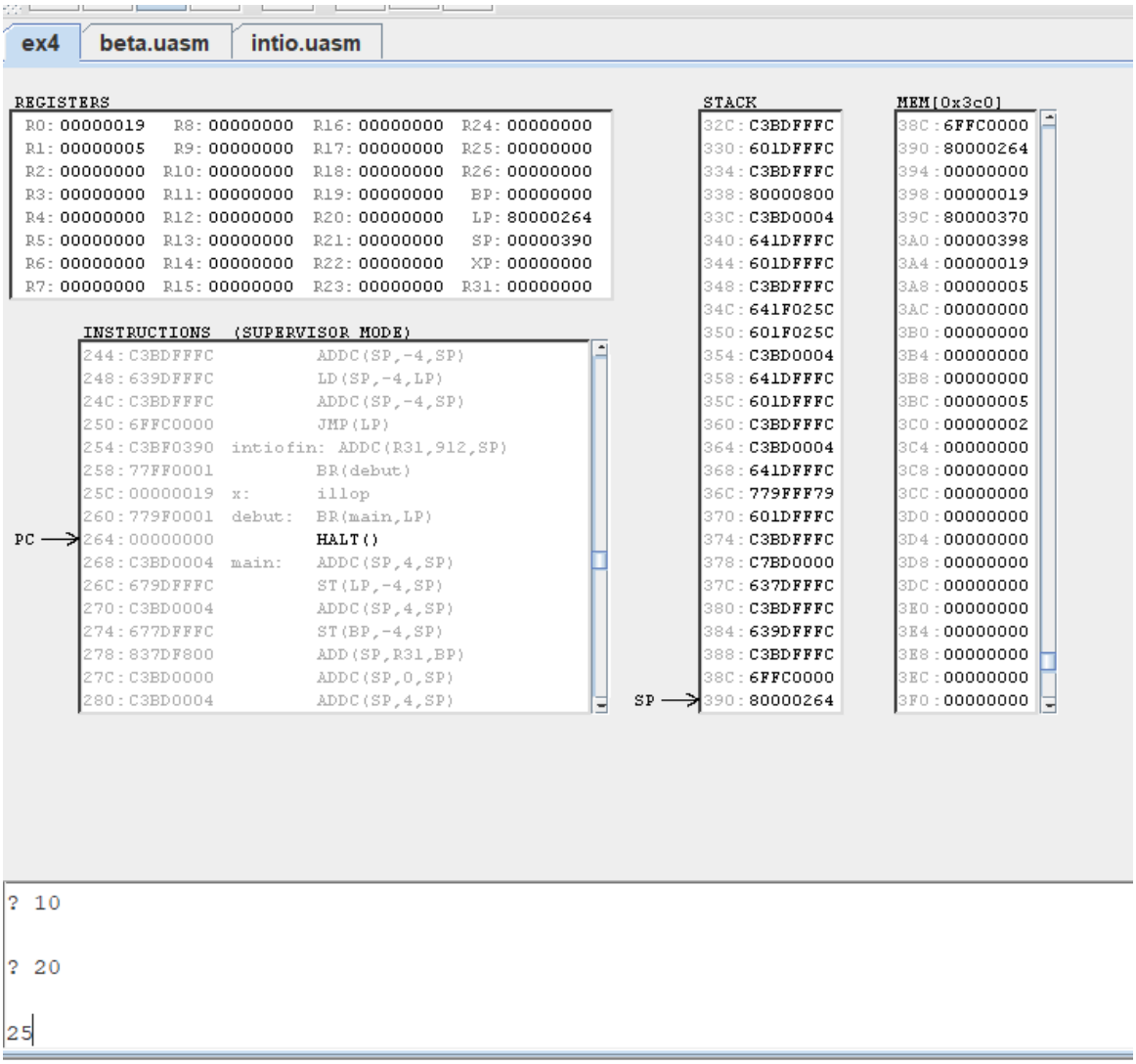
2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
 └─FONCTION/main
    └─AFF
       └─IDF/x
       └─PLUS
          └─MUL
             └─LIRE
             └─CONST/2
          └─DIV
             └─MOINS
                └─LIRE
                └─CONST/5
          └─CONST/3
    └─ECR
       └─IDF/x

========== TABLE DES SYMBOLES ==========
```

```
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=x, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        x : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        RDINT()
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        RDINT()
        PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
        LD(x, R0)
        PUSH(R0)
        POP(R0)
```

```
        WRINT()
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Le code généré respecte la structure attendue : prologue et épilogue corrects, gestion conforme de la pile, utilisation des macros *RDINT()* et *WRINT()* calcul correct de l'expression arithmétique et stockage approprié de la variable globale. L'exécution dans BSim produit le résultat attendu pour les valeurs saisies, ce qui valide la génération de code.

La vérification :

Calcul mathématique (première lecture : 10, deuxième lecture : 20) :

1. x = 10 * 2 + (20 - 5) / 3
2. x = 20 + 15 / 3
3. x = 20 + 5
4. x = 25

Après exécution complète du programme dans BSim, on obtient :

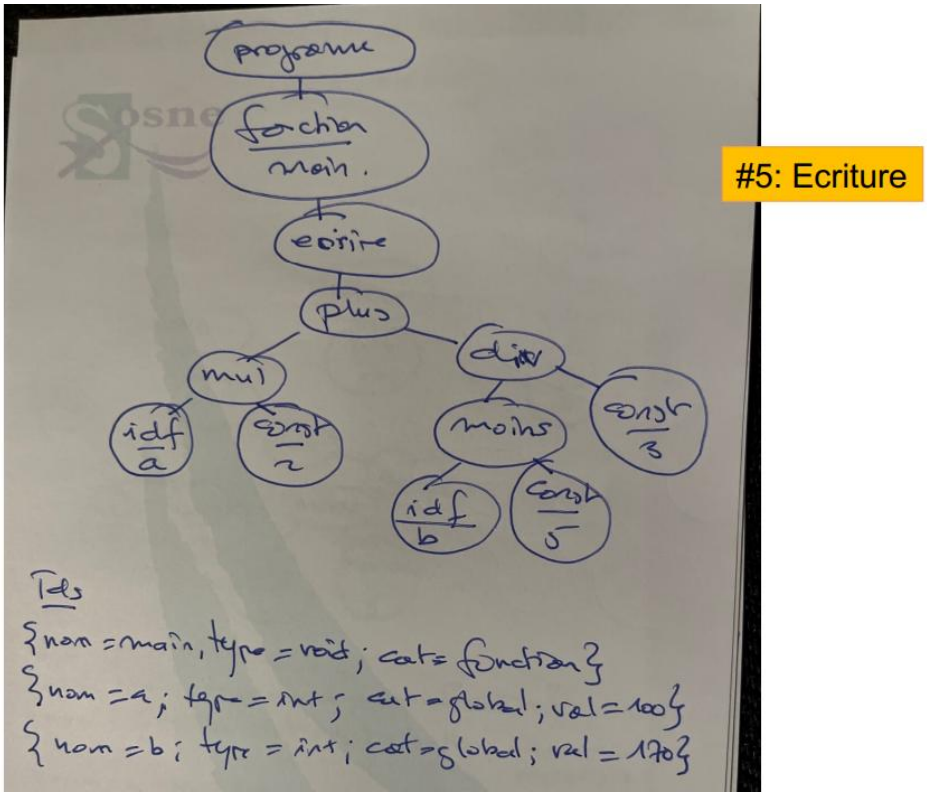
x (0x25C) = 00000019

La valeur 00000019 en hexadécimal correspond à 25 en décimal => La valeur calculée et stockée en mémoire correspond exactement au résultat mathématique attendu.

Exemple 5 : Ecriture

1. Condition du test

Programme composé d'une seule fonction *main* sans paramètres ni variables locales. La fonction contient uniquement une instruction d'écriture : *ecrire(a∗2+(b−5)/3)*. Les variables *a* et *b* sont globales et initialisées respectivement à 100 et 170 dans la mémoire statique.



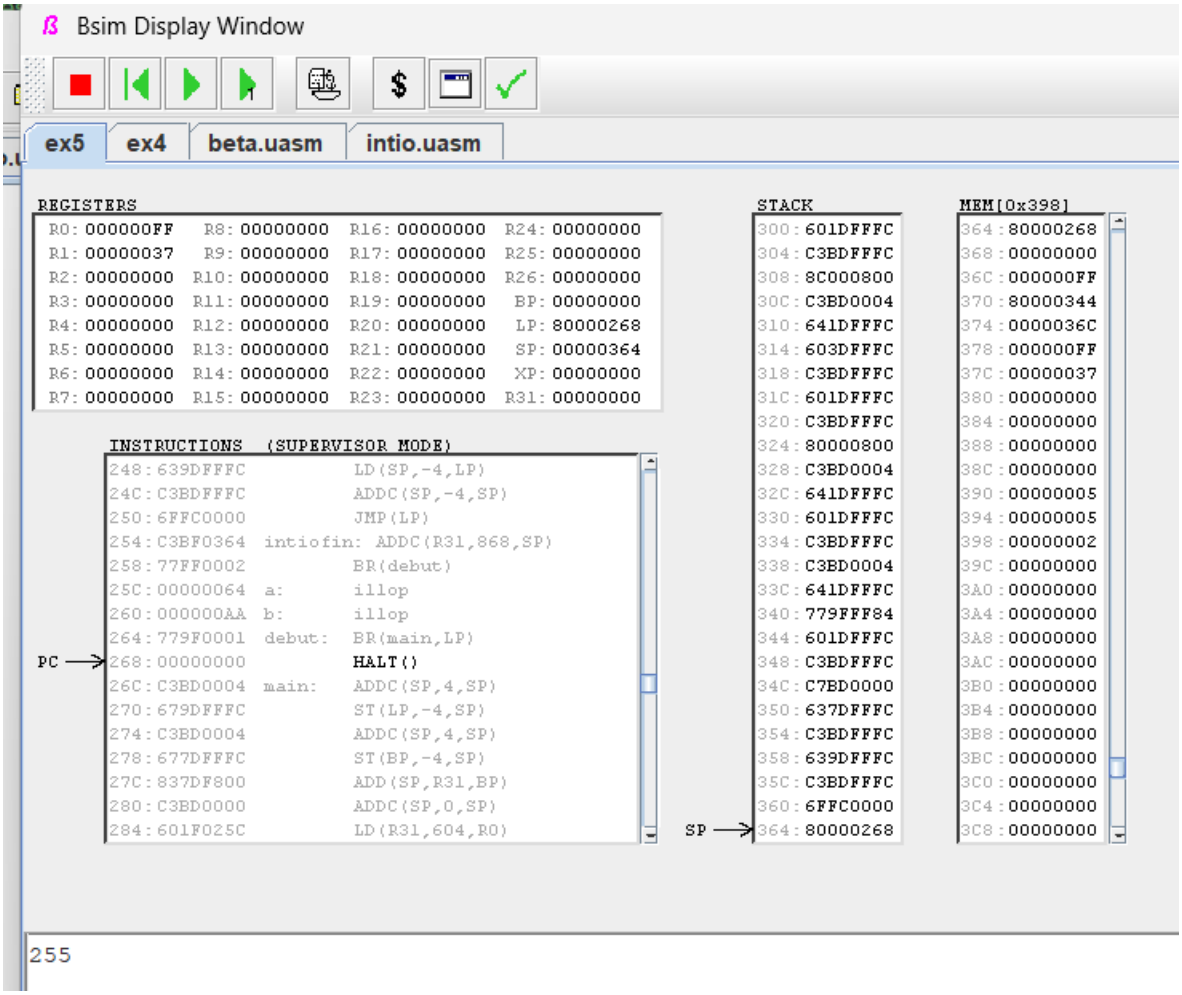
2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
   └─ECR
      └─PLUS
         └─MUL
            └─IDF/a
            └─CONST/2
      └─DIV
         └─MOINS
            └─IDF/b
            └─CONST/5
         └─CONST/3

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=a, type=int, cat=GLOBAL, val=100 }
```

```
{ nom=b, type=int, cat=GLOBAL, val=170 }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        a : LONG(100)
        b : LONG(170)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        LD(a, R0)
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        LD(b, R0)
        PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        WRINT()
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
```

```
        RTN()
pile:
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Le code généré respecte la structure de l'arbre abstrait et la priorité des opérations.

Vérification mathématique :

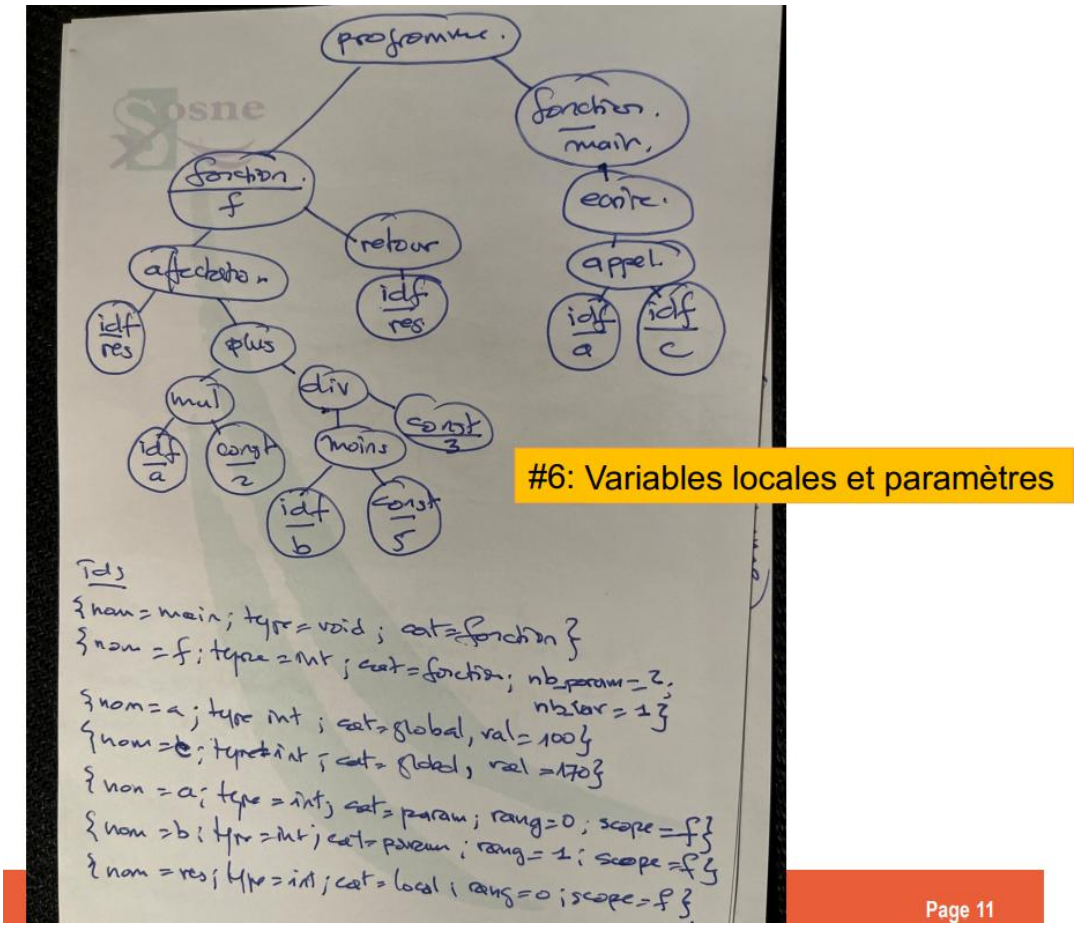
1. a * 2 = 100 × 2 = 200
2. b - 5 = 170 - 5 = 165
3. 165 / 3 = 55
4. 200 + 55 = 255

Le résultat affiché dans BSim est 255, ce qui correspond exactement au résultat mathématique attendu.

Exemple 6 : Variables locales et paramètres

1. Condition du test

Programme composé de deux fonctions : *f* et *main*. La fonction *f* possède deux paramètres *(a, b)* et une variable locale *(res)*. Elle calcule l'expression *a*2 + (b−5)/3*, stocke le résultat dans *res*, puis retourne cette valeur. La fonction *main* appelle *f(a, c)* et affiche le résultat avec *ecrire*. Les variables *a* et *c* sont globales et initialisées respectivement à 100 et 170.



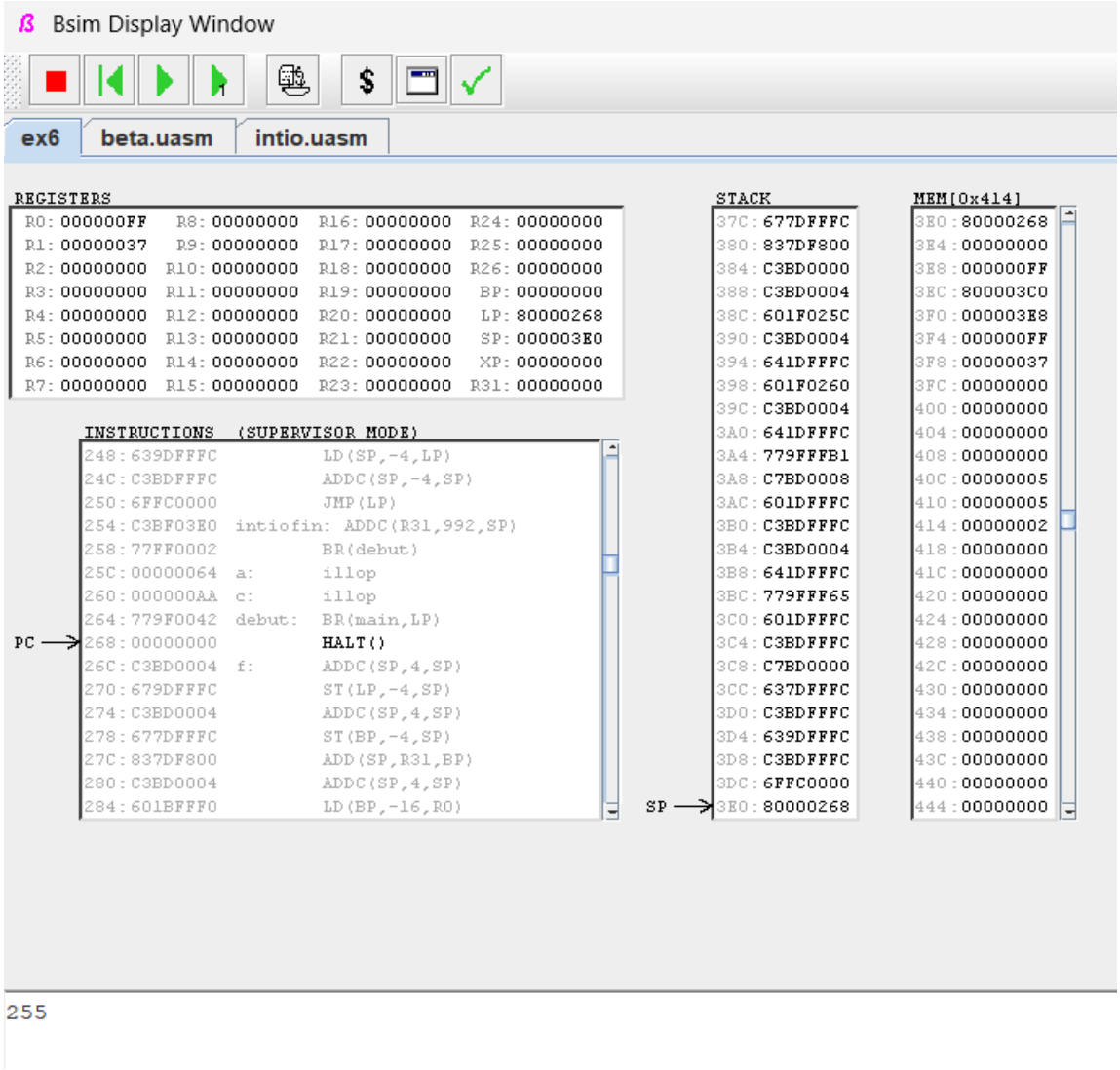
2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
 └─FONCTION/f
    └─AFF
       └─IDF/res
       └─PLUS
          └─MUL
             └─IDF/a
             └─CONST/2
          └─DIV
             └─MOINS
                └─IDF/b
                └─CONST/5
```

```
                └─CONST/3
    └─RET/f
        └─IDF/res
└─FONCTION/main
    └─ECR
        └─APPEL/f
            └─IDF/a
            └─IDF/c

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=f, type=int, cat=FONCTION, nb_param=2, nb_var=1 }
{ nom=a, type=int, cat=GLOBAL, val=100 }
{ nom=c, type=int, cat=GLOBAL, val=170 }
{ nom=a@f, type=int, cat=PARAM, rang=0, scope=f }
{ nom=b@f, type=int, cat=PARAM, rang=1, scope=f }
{ nom=res@f, type=int, cat=LOCAL, rang=0, scope=f }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        a : LONG(100)
        c : LONG(170)
debut:
        CALL(main)
        HALT()
f:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(1)
        GETFRAME(-16, R0)
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        GETFRAME(-12, R0)
        PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
```

```
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, 0)
        GETFRAME(0, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, -20)
        BR(ret_f)
ret_f:
        DEALLOCATE(1)
        POP(BP)
        POP(LP)
        RTN()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        ALLOCATE(1)
        LD(a, R0)
        PUSH(R0)
        LD(c, R0)
        PUSH(R0)
        CALL(f)
        DEALLOCATE(2)
        POP(R0)
        WRINT()
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Lors du premier test dans BSim, une erreur « ST to invalid memory location » est apparue. Cette erreur provenait de l'absence d'une zone mémoire réservée pour la pile. Le problème a été corrigé en ajoutant à la fin du programme : *pile: STORAGE(200).* Après cette correction le programme s'exécute correctement.

L'exécution dans BSim affiche 255, ce qui correspond au calcul attendu :

1. a * 2 = 100 × 2 = 200
2. b - 5 = 170 - 5 = 165
3. 165 / 3 = 55
4. 200 + 55 = 255

Dans notre TDS, les variables locales et paramètres sont notés sous la forme *a@f*, *b@f*, *res@f* afin de représenter explicitement leur portée (scope) et éviter toute confusion avec les variables globales. Le code généré respecte donc la gestion correcte des paramètres, des variables locales et de la valeur de retour conformément au modèle vu en cours.

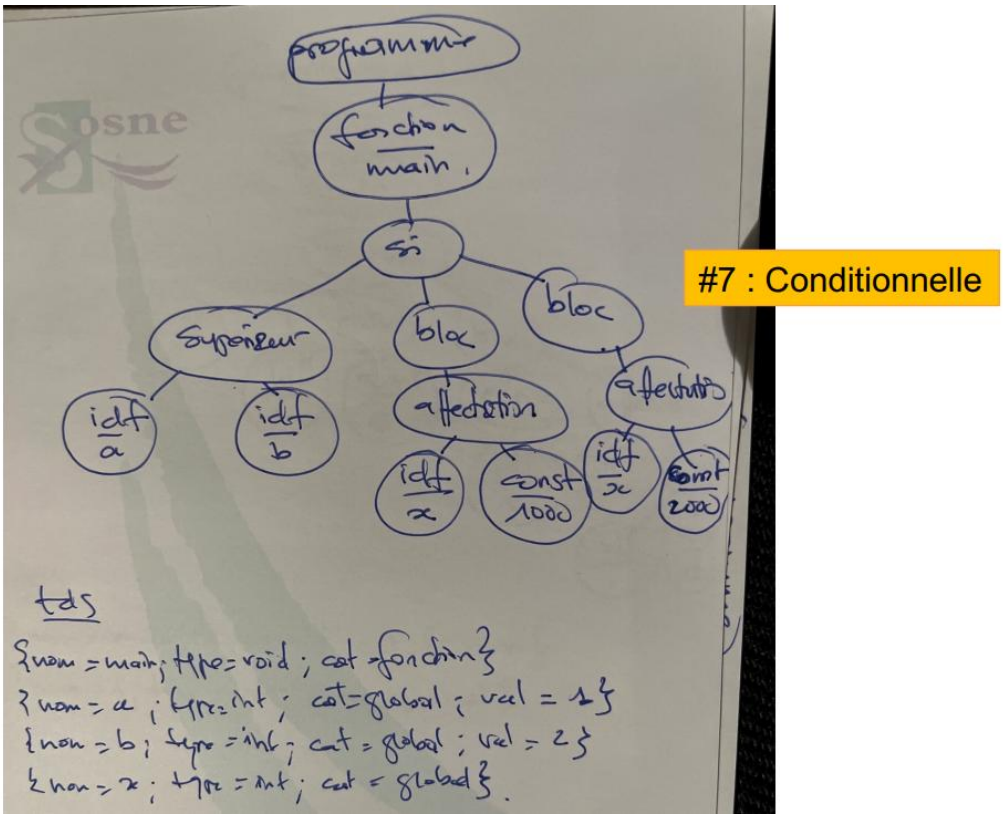
Exemple 7 : Conditionnelle

1. Condition du test

Programme composé d'une seule fonction *main*. La fonction contient une instruction conditionnelle :

si (a > b) alors
  x ← 1000
sinon
  x ← 2000

Les variables *a*, *b* et *x* sont globales. Les valeurs initiales sont : a = 1; b = 2 ; x = 0.



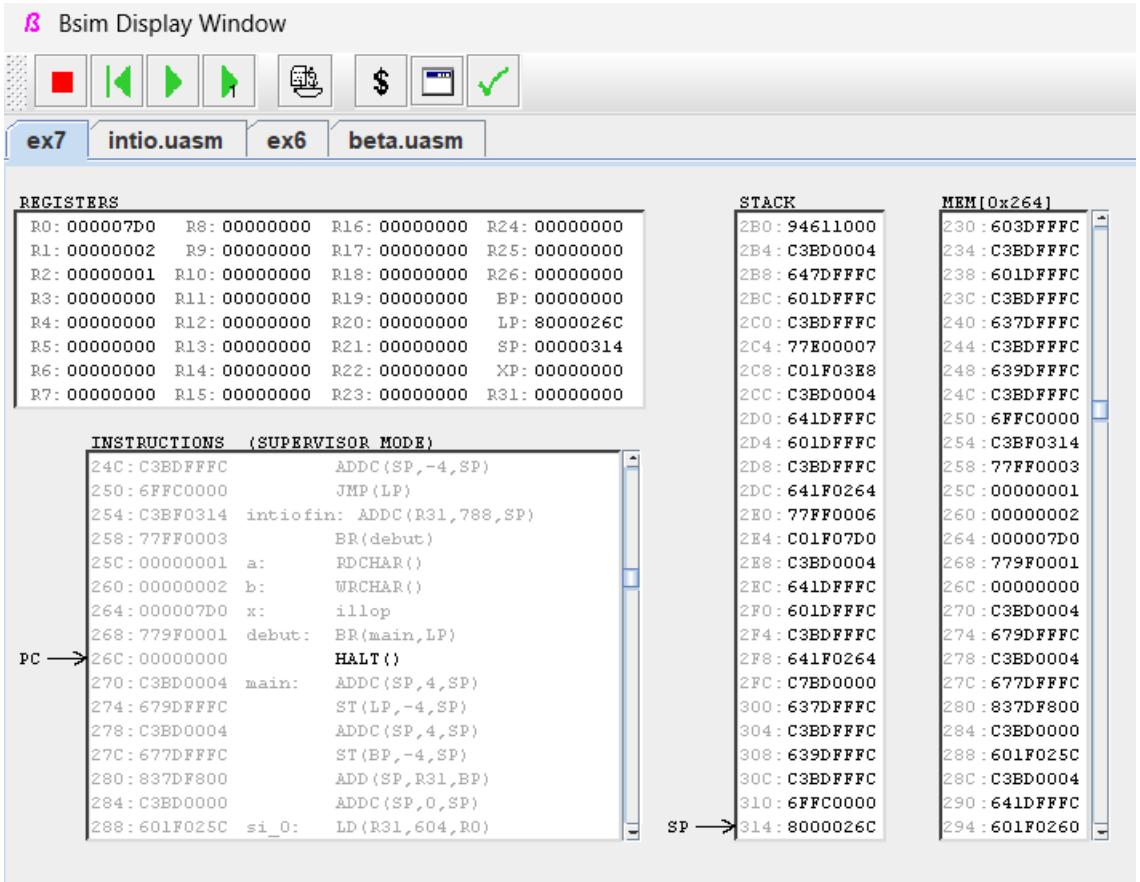
2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
 └─FONCTION/main
    └─SI/0
       └─SUP
          └─IDF/a
          └─IDF/b
       └─BLOC
          └─AFF
             └─IDF/x
             └─CONST/1000
       └─BLOC
          └─AFF
```

```
                └─IDF/x
                └─CONST/2000

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=a, type=int, cat=GLOBAL, val=1 }
{ nom=b, type=int, cat=GLOBAL, val=2 }
{ nom=x, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        a : LONG(1)
        b : LONG(2)
        x : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
si_0:
        LD(a, R0)
        PUSH(R0)
        LD(b, R0)
        PUSH(R0)
        POP(R1)
        POP(R2)
        CMPLT(R1, R2, R3)
        PUSH(R3)
        POP(R0)
        BF(R0, sinon_0)
        CMOVE(1000, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
        BR(fsi_0)
sinon_0:
        CMOVE(2000, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
fsi_0:
ret_main:
```

```
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Le code généré respecte correctement la structure d'une instruction conditionnelle (BF, labels sinon, saut vers fin). La gestion de la pile est cohérente et l'affectation dans la variable globale *x* est correctement réalisée.

Dans BSim, on observe après exécution :

x (0x264) = 000007D0
R0 = 000007D0

0x7D0 en hexadécimal correspond à 2000 en décimal. Comme la condition $a > b$ ($1 > 2$) est fausse, le bloc sinon est exécuté et *x* reçoit la valeur 2000 => La valeur stockée en mémoire correspond donc exactement au résultat attendu.

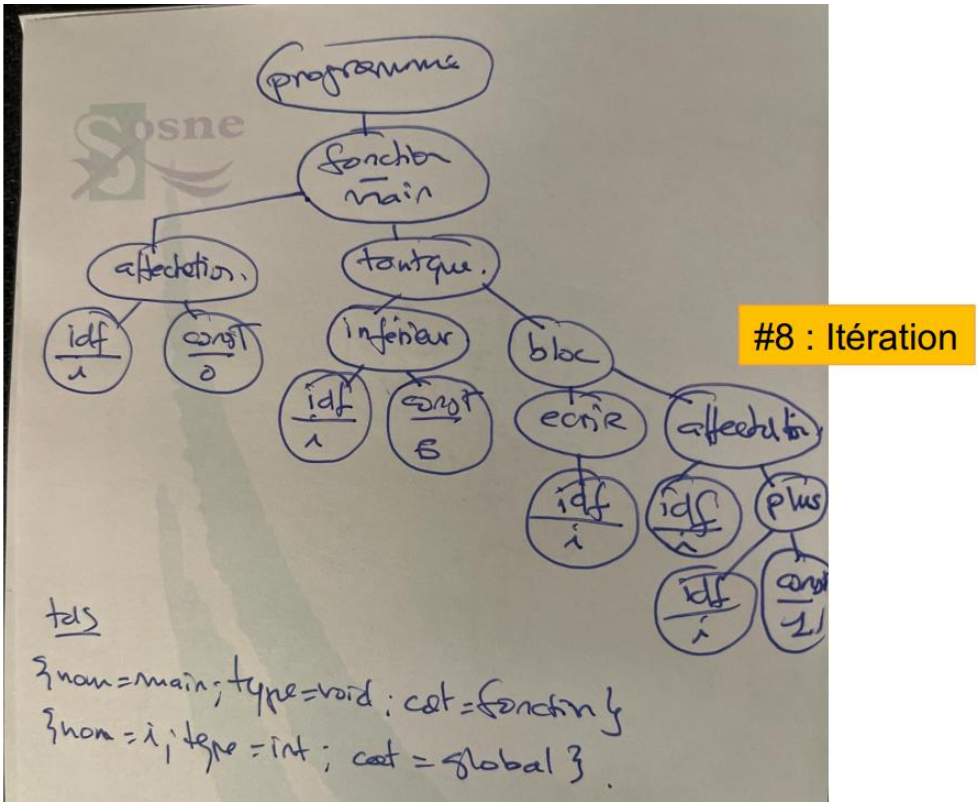
Exemple 8 : Itération

1. Condition du test

Le programme contient une seule fonction *main*. La variable globale *i* est initialisée à 0. Une boucle *tant que* est exécutée tant que la condition $i < 6$ est vraie. À chaque itération :

1. La valeur de *i* est affichée avec *WRINT()*.
2. La variable *i* est incrémentée de 1 ($i = i + 1$).

Le programme doit donc afficher les valeurs de 0 à 5.



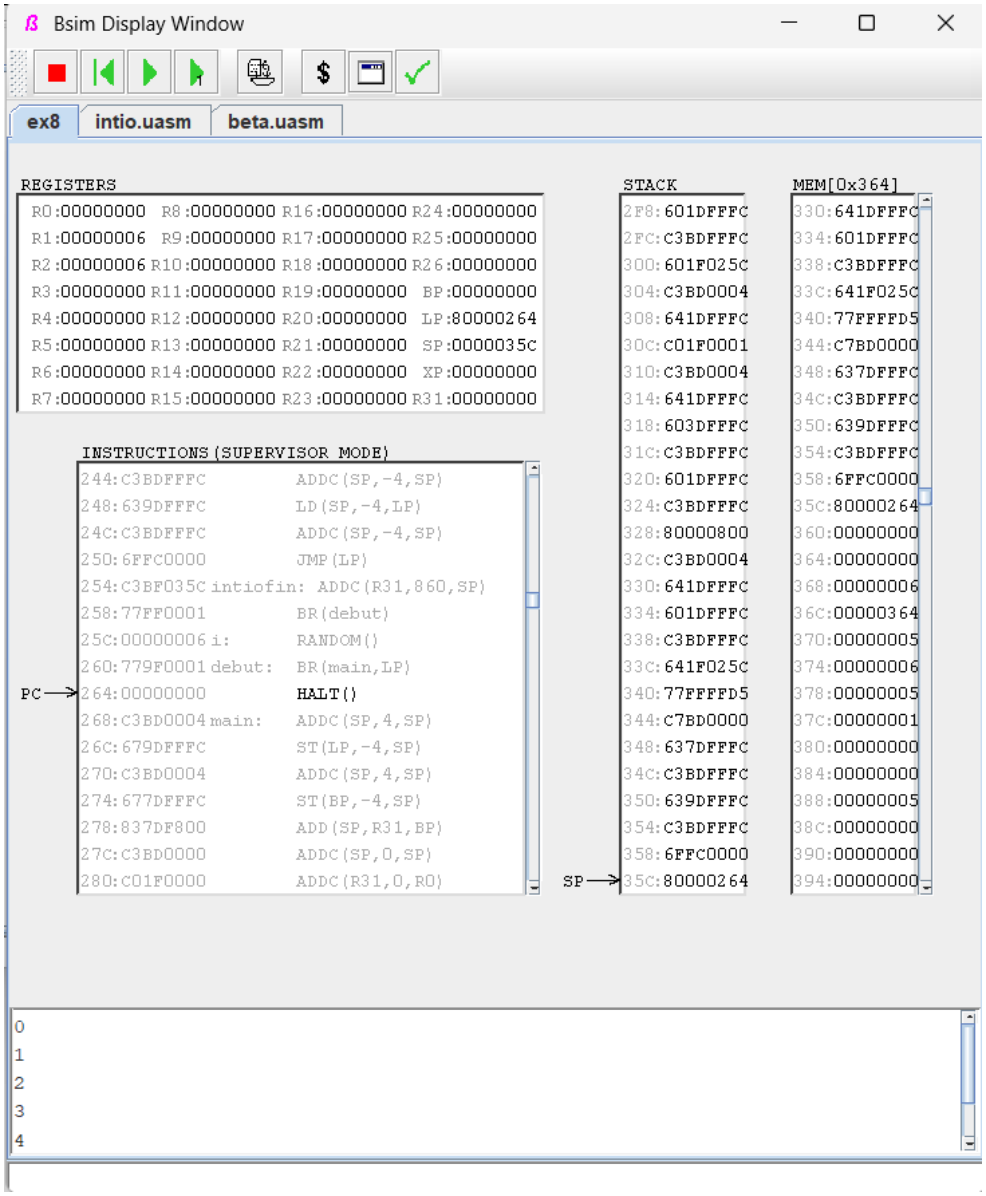
2. Résultat de notre programme

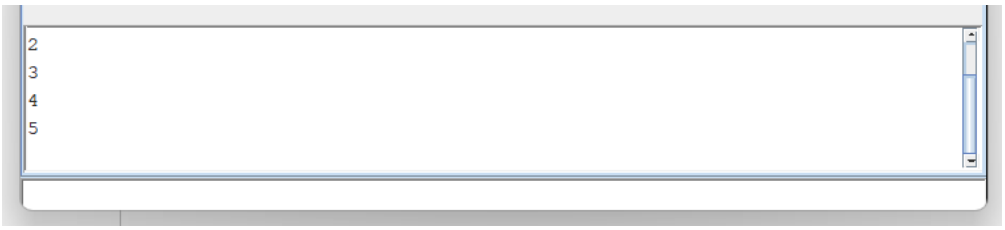
```
========== ARBRE ABSTRAIT ==========
PROG
 └─FONCTION/main
    └─AFF
       └─IDF/i
       └─CONST/0
    └─TQ/0
       └─INF
          └─IDF/i
          └─CONST/6
       └─BLOC
          └─ECR
             └─IDF/i
          └─AFF
```

```
                └─IDF/i
                └─PLUS
                    └─IDF/i
                    └─CONST/1

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=i, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        i : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        CMOVE(0, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, i)
tq_0:
        LD(i, R0)
        PUSH(R0)
        CMOVE(6, R0)
        PUSH(R0)
        POP(R1)
        POP(R2)
        CMPLT(R2, R1, R3)
        PUSH(R3)
        POP(R0)
        BF(R0, ftq_0)
        LD(i, R0)
        PUSH(R0)
        POP(R0)
        WRINT()
        LD(i, R0)
        PUSH(R0)
        CMOVE(1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
```

```
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, i)
        BR(tq_0)
ftq_0:
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)
```

3. Exécution avec BSim





4. Évaluation : comparaison avec le code attendu

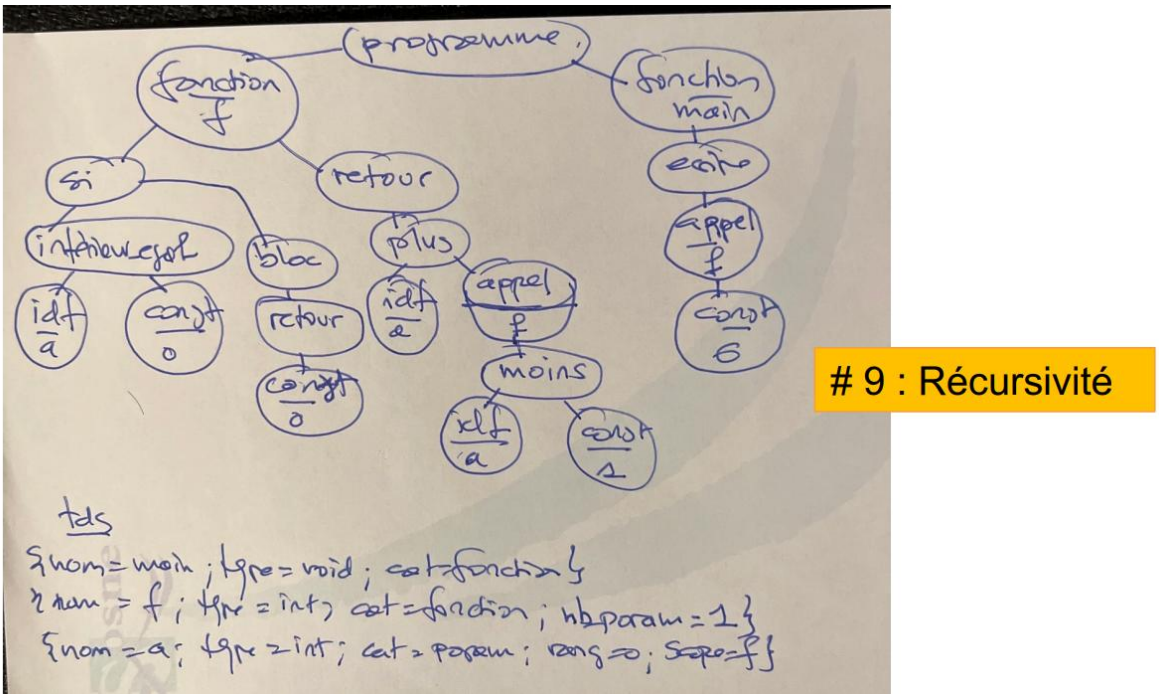
Le code généré respecte la structure de l'arbre abstrait. L'initialisation de la variable globale *i* est correcte, la condition *i < 6* est bien évaluée à chaque itération grâce à l'instruction *CMPLT*, et le branchement *BF* permet de sortir correctement de la boucle lorsque la condition devient fausse. La gestion de la pile est cohérente (*PUSH*/*POP*), l'instruction *ADD* assure correctement l'incrémentation de *i*, et l'affichage via *WRINT()* produit les valeurs attendues.

L'exécution dans BSim confirme que le programme affiche successivement 0, 1, 2, 3, 4 et 5, puis s'arrête lorsque *i* atteint 6. La valeur finale en mémoire correspond donc exactement à l'attendu.

Exemple 9 : Récursivité

1. Condition du test

Le programme contient deux fonctions : *f* et *main*. La fonction *f* est récursive et possède un paramètre *a*. Elle retourne 0 si $a \leq 0$, sinon elle retourne $a + f(a - 1)$. Il n'y a pas de variables locales. La fonction main appelle *f(6)* et affiche le résultat avec *ecrire*.



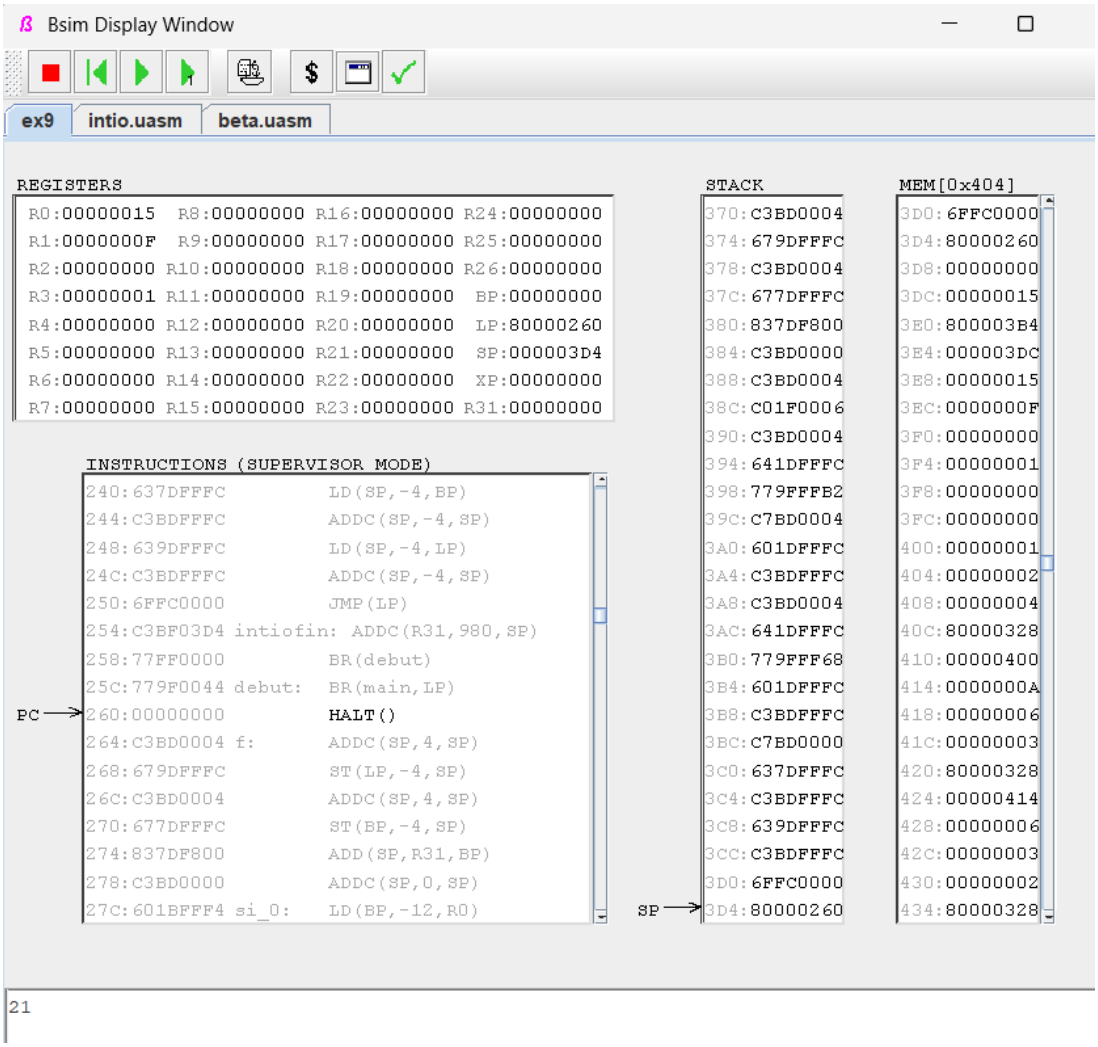
2. Résultat de notre programme

```
========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/f
  └─SI/0
    └─INFE
      └─IDF/a
      └─CONST/0
    └─BLOC
      └─RET/f
        └─CONST/0
    └─BLOC
  └─RET/f
    └─PLUS
      └─IDF/a
      └─APPEL/f
        └─MOINS
          └─IDF/a
          └─CONST/1
└─FONCTION/main
  └─ECR
```

```
    └─APPEL/f
      └─CONST/6

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=f, type=int, cat=FONCTION, nb_param=1 }
{ nom=a@f, type=int, cat=PARAM, rang=0, scope=f }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
debut:
        CALL(main)
        HALT()
f:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
si_0:
        GETFRAME(-12, R0)
        PUSH(R0)
        CMOVE(0, R0)
        PUSH(R0)
        POP(R1)
        POP(R2)
        CMPLT(R1, R2, R3)
        CMPEQ(R3, R31, R3)
        PUSH(R3)
        POP(R0)
        BF(R0, sinon_0)
        CMOVE(0, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, -16)
        BR(ret_f)
        BR(fsi_0)
sinon_0:
fsi_0:
        GETFRAME(-12, R0)
        PUSH(R0)
        ALLOCATE(1)
        GETFRAME(-12, R0)
        PUSH(R0)
        CMOVE(1, R0)
        PUSH(R0)
```

```
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CALL(f)
        DEALLOCATE(1)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, -16)
        BR(ret_f)
ret_f:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        ALLOCATE(1)
        CMOVE(6, R0)
        PUSH(R0)
        CALL(f)
        DEALLOCATE(1)
        POP(R0)
        WRINT()
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)
```

3. Exécution avec BSim



4. Évaluation : comparaison avec le code attendu

Le code généré fonctionne correctement. La condition $a \leq 0$ est bien testée et la fonction s'arrête au bon moment. L'appel récursif $f(a - 1)$ est bien exécuté et chaque appel crée un nouveau frame dans la pile, puis la pile est correctement restaurée après le retour.

Dans BSim, le programme affiche 21 (00000015 en hexadécimal), ce qui correspond exactement au résultat attendu pour $f(6)$. On peut donc conclure que la récursivité et la gestion de la pile sont correctes.

Conclusion

Lors de cette première étape, nous avons mis en place les bases de notre compilateur en développant les packages TDS, GENERATION ainsi que les différentes classes de tests.

Nous avons implémenté la gestion de la table des symboles, la génération du code assembleur BETA et validé progressivement chaque fonctionnalité à l'aide d'exemples exécutés sous BSim.